

```
#!/usr/bin/env python3
"""Generate and serve a review page for eval results.

Reads the workspace directory, discovers runs (directories with outputs/),
embeds all output data into a self-contained HTML page, and serves it via
a tiny HTTP server. Feedback auto-saves to feedback.json in the workspace.

Usage:
    python generate_review.py <workspace-path> [--port PORT] [--skill-name NAME]
    python generate_review.py <workspace-path> --previous-feedback
/path/to/old/feedback.json

No dependencies beyond the Python stdlib are required.
"""

import argparse
import base64
import json
import mimetypes
import os
import re
import signal
import subprocess
import sys
import time
import webbrowser
from functools import partial
from http.server import HTTPServer, BaseHTTPRequestHandler
from pathlib import Path

# Files to exclude from output listings
METADATA_FILES = {"transcript.md", "user_notes.md", "metrics.json"}

# Extensions we render as inline text
TEXT_EXTENSIONS = {
    ".txt", ".md", ".json", ".csv", ".py", ".js", ".ts", ".tsx", ".jsx",
    ".yaml", ".yml", ".xml", ".html", ".css", ".sh", ".rb", ".go", ".rs",
    ".java", ".c", ".cpp", ".h", ".hpp", ".sql", ".r", ".toml",
}

# Extensions we render as inline images
IMAGE_EXTENSIONS = {".png", ".jpg", ".jpeg", ".gif", ".svg", ".webp"}

# MIME type overrides for common types
MIME_OVERRIDES = {
    ".svg": "image/svg+xml",
    ".xlsx": "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
    ".docx": "application/vnd.openxmlformats-officedocument.wordprocessingml.document",
    ".pptx":
"application/vnd.openxmlformats-officedocument.presentationml.presentation",
}

def get_mime_type(path: Path) -> str:
    ext = path.suffix.lower()
```

```

    if ext in MIME_OVERRIDES:
        return MIME_OVERRIDES[ext]
    mime, _ = mimetypes.guess_type(str(path))
    return mime or "application/octet-stream"

def find_runs(workspace: Path) -> list[dict]:
    """Recursively find directories that contain an outputs/ subdirectory."""
    runs: list[dict] = []
    _find_runs_recursive(workspace, workspace, runs)
    runs.sort(key=lambda r: (r.get("eval_id", float("inf")), r["id"]))
    return runs

def _find_runs_recursive(root: Path, current: Path, runs: list[dict]) -> None:
    if not current.is_dir():
        return

    outputs_dir = current / "outputs"
    if outputs_dir.is_dir():
        run = build_run(root, current)
        if run:
            runs.append(run)
        return

    skip = {"node_modules", ".git", "__pycache__", "skill", "inputs"}
    for child in sorted(current.iterdir()):
        if child.is_dir() and child.name not in skip:
            _find_runs_recursive(root, child, runs)

def build_run(root: Path, run_dir: Path) -> dict | None:
    """Build a run dict with prompt, outputs, and grading data."""
    prompt = ""
    eval_id = None

    # Try eval_metadata.json
    for candidate in [run_dir / "eval_metadata.json", run_dir.parent /
"eval_metadata.json"]:
        if candidate.exists():
            try:
                metadata = json.loads(candidate.read_text())
                prompt = metadata.get("prompt", "")
                eval_id = metadata.get("eval_id")
            except (json.JSONDecodeError, OSError):
                pass
            if prompt:
                break

    # Fall back to transcript.md
    if not prompt:
        for candidate in [run_dir / "transcript.md", run_dir / "outputs" /
"transcript.md"]:
            if candidate.exists():
                try:

```

```

        text = candidate.read_text()
        match = re.search(r"## Eval Prompt\n\n([\s\S]*) (?:\n##|$)", text)
        if match:
            prompt = match.group(1).strip()
        except OSError:
            pass
        if prompt:
            break

if not prompt:
    prompt = "(No prompt found)"

run_id = str(run_dir.relative_to(root)).replace("/", "-").replace("\\", "-")

# Collect output files
outputs_dir = run_dir / "outputs"
output_files: list[dict] = []
if outputs_dir.is_dir():
    for f in sorted(outputs_dir.iterdir()):
        if f.is_file() and f.name not in METADATA_FILES:
            output_files.append(embed_file(f))

# Load grading if present
grading = None
for candidate in [run_dir / "grading.json", run_dir.parent / "grading.json"]:
    if candidate.exists():
        try:
            grading = json.loads(candidate.read_text())
        except (json.JSONDecodeError, OSError):
            pass
        if grading:
            break

return {
    "id": run_id,
    "prompt": prompt,
    "eval_id": eval_id,
    "outputs": output_files,
    "grading": grading,
}

def embed_file(path: Path) -> dict:
    """Read a file and return an embedded representation."""
    ext = path.suffix.lower()
    mime = get_mime_type(path)

    if ext in TEXT_EXTENSIONS:
        try:
            content = path.read_text(errors="replace")
        except OSError:
            content = "(Error reading file)"
        return {
            "name": path.name,
            "type": "text",

```

```

        "content": content,
    }
elif ext in IMAGE_EXTENSIONS:
    try:
        raw = path.read_bytes()
        b64 = base64.b64encode(raw).decode("ascii")
    except OSError:
        return {"name": path.name, "type": "error", "content": "(Error reading
file)"}
    return {
        "name": path.name,
        "type": "image",
        "mime": mime,
        "data_uri": f"data:{mime};base64,{b64}",
    }
elif ext == ".pdf":
    try:
        raw = path.read_bytes()
        b64 = base64.b64encode(raw).decode("ascii")
    except OSError:
        return {"name": path.name, "type": "error", "content": "(Error reading
file)"}
    return {
        "name": path.name,
        "type": "pdf",
        "data_uri": f"data:{mime};base64,{b64}",
    }
elif ext == ".xlsx":
    try:
        raw = path.read_bytes()
        b64 = base64.b64encode(raw).decode("ascii")
    except OSError:
        return {"name": path.name, "type": "error", "content": "(Error reading
file)"}
    return {
        "name": path.name,
        "type": "xlsx",
        "data_b64": b64,
    }
else:
    # Binary / unknown - base64 download link
    try:
        raw = path.read_bytes()
        b64 = base64.b64encode(raw).decode("ascii")
    except OSError:
        return {"name": path.name, "type": "error", "content": "(Error reading
file)"}
    return {
        "name": path.name,
        "type": "binary",
        "mime": mime,
        "data_uri": f"data:{mime};base64,{b64}",
    }
}

```

```

def load_previous_iteration(workspace: Path) -> dict[str, dict]:
    """Load previous iteration's feedback and outputs.

    Returns a map of run_id -> {"feedback": str, "outputs": list[dict]}.
    """
    result: dict[str, dict] = {}

    # Load feedback
    feedback_map: dict[str, str] = {}
    feedback_path = workspace / "feedback.json"
    if feedback_path.exists():
        try:
            data = json.loads(feedback_path.read_text())
            feedback_map = {
                r["run_id"]: r["feedback"]
                for r in data.get("reviews", [])
                if r.get("feedback", "").strip()
            }
        except (json.JSONDecodeError, OSError, KeyError):
            pass

    # Load runs (to get outputs)
    prev_runs = find_runs(workspace)
    for run in prev_runs:
        result[run["id"]] = {
            "feedback": feedback_map.get(run["id"], ""),
            "outputs": run.get("outputs", []),
        }

    # Also add feedback for run_ids that had feedback but no matching run
    for run_id, fb in feedback_map.items():
        if run_id not in result:
            result[run_id] = {"feedback": fb, "outputs": []}

    return result

def generate_html(
    runs: list[dict],
    skill_name: str,
    previous: dict[str, dict] | None = None,
    benchmark: dict | None = None,
) -> str:
    """Generate the complete standalone HTML page with embedded data."""
    template_path = Path(__file__).parent / "viewer.html"
    template = template_path.read_text()

    # Build previous_feedback and previous_outputs maps for the template
    previous_feedback: dict[str, str] = {}
    previous_outputs: dict[str, list[dict]] = {}
    if previous:
        for run_id, data in previous.items():
            if data.get("feedback"):
                previous_feedback[run_id] = data["feedback"]
            if data.get("outputs"):

```

```

        previous_outputs[run_id] = data["outputs"]

    embedded = {
        "skill_name": skill_name,
        "runs": runs,
        "previous_feedback": previous_feedback,
        "previous_outputs": previous_outputs,
    }
    if benchmark:
        embedded["benchmark"] = benchmark

    data_json = json.dumps(embedded)

    return template.replace("/*__EMBEDDED_DATA__*/", f"const EMBEDDED_DATA = {data_json};")

# -----
# HTTP server (stdlib only, zero dependencies)
# -----

def _kill_port(port: int) -> None:
    """Kill any process listening on the given port."""
    try:
        result = subprocess.run(
            ["lsof", "-ti", f":{port}"],
            capture_output=True, text=True, timeout=5,
        )
        for pid_str in result.stdout.strip().split("\n"):
            if pid_str.strip():
                try:
                    os.kill(int(pid_str.strip()), signal.SIGTERM)
                except (ProcessLookupError, ValueError):
                    pass
        if result.stdout.strip():
            time.sleep(0.5)
    except subprocess.TimeoutExpired:
        pass
    except FileNotFoundError:
        print("Note: lsof not found, cannot check if port is in use", file=sys.stderr)

class ReviewHandler(BaseHTTPRequestHandler):
    """Serves the review HTML and handles feedback saves.

    Regenerates the HTML on each page load so that refreshing the browser
    picks up new eval outputs without restarting the server.
    """

    def __init__(
        self,
        workspace: Path,
        skill_name: str,
        feedback_path: Path,
        previous: dict[str, dict],
        benchmark_path: Path | None,
    ):

```

```

        *args,
        **kwargs,
    ):
        self.workspace = workspace
        self.skill_name = skill_name
        self.feedback_path = feedback_path
        self.previous = previous
        self.benchmark_path = benchmark_path
        super().__init__(*args, **kwargs)

def do_GET(self) -> None:
    if self.path == "/" or self.path == "/index.html":
        # Regenerate HTML on each request (re-scans workspace for new outputs)
        runs = find_runs(self.workspace)
        benchmark = None
        if self.benchmark_path and self.benchmark_path.exists():
            try:
                benchmark = json.loads(self.benchmark_path.read_text())
            except (json.JSONDecodeError, OSError):
                pass
        html = generate_html(runs, self.skill_name, self.previous, benchmark)
        content = html.encode("utf-8")
        self.send_response(200)
        self.send_header("Content-Type", "text/html; charset=utf-8")
        self.send_header("Content-Length", str(len(content)))
        self.end_headers()
        self.wfile.write(content)
    elif self.path == "/api/feedback":
        data = b"{}"
        if self.feedback_path.exists():
            data = self.feedback_path.read_bytes()
        self.send_response(200)
        self.send_header("Content-Type", "application/json")
        self.send_header("Content-Length", str(len(data)))
        self.end_headers()
        self.wfile.write(data)
    else:
        self.send_error(404)

def do_POST(self) -> None:
    if self.path == "/api/feedback":
        length = int(self.headers.get("Content-Length", 0))
        body = self.rfile.read(length)
        try:
            data = json.loads(body)
            if not isinstance(data, dict) or "reviews" not in data:
                raise ValueError("Expected JSON object with 'reviews' key")
            self.feedback_path.write_text(json.dumps(data, indent=2) + "\n")
            resp = b'{"ok":true}'
            self.send_response(200)
        except (json.JSONDecodeError, OSError, ValueError) as e:
            resp = json.dumps({"error": str(e)}).encode()
            self.send_response(500)
        self.send_header("Content-Type", "application/json")
        self.send_header("Content-Length", str(len(resp)))

```

```

        self.end_headers()
        self.wfile.write(resp)
    else:
        self.send_error(404)

def log_message(self, format: str, *args: object) -> None:
    # Suppress request logging to keep terminal clean
    pass

def main() -> None:
    parser = argparse.ArgumentParser(description="Generate and serve eval review")
    parser.add_argument("workspace", type=Path, help="Path to workspace directory")
    parser.add_argument("--port", "-p", type=int, default=3117, help="Server port (default: 3117)")
    parser.add_argument("--skill-name", "-n", type=str, default=None, help="Skill name for header")
    parser.add_argument(
        "--previous-workspace", type=Path, default=None,
        help="Path to previous iteration's workspace (shows old outputs and feedback as context)",
    )
    parser.add_argument(
        "--benchmark", type=Path, default=None,
        help="Path to benchmark.json to show in the Benchmark tab",
    )
    parser.add_argument(
        "--static", "-s", type=Path, default=None,
        help="Write standalone HTML to this path instead of starting a server",
    )
    args = parser.parse_args()

    workspace = args.workspace.resolve()
    if not workspace.is_dir():
        print(f"Error: {workspace} is not a directory", file=sys.stderr)
        sys.exit(1)

    runs = find_runs(workspace)
    if not runs:
        print(f"No runs found in {workspace}", file=sys.stderr)
        sys.exit(1)

    skill_name = args.skill_name or workspace.name.replace("-workspace", "")
    feedback_path = workspace / "feedback.json"

    previous: dict[str, dict] = {}
    if args.previous_workspace:
        previous = load_previous_iteration(args.previous_workspace.resolve())

    benchmark_path = args.benchmark.resolve() if args.benchmark else None
    benchmark = None
    if benchmark_path and benchmark_path.exists():
        try:
            benchmark = json.loads(benchmark_path.read_text())
        except (json.JSONDecodeError, OSError):

```



```

        pass

    if args.static:
        html = generate_html(runs, skill_name, previous, benchmark)
        args.static.parent.mkdir(parents=True, exist_ok=True)
        args.static.write_text(html)
        print(f"\n Static viewer written to: {args.static}\n")
        sys.exit(0)

    # Kill any existing process on the target port
    port = args.port
    _kill_port(port)
    handler = partial(ReviewHandler, workspace, skill_name, feedback_path, previous,
benchmark_path)
    try:
        server = HTTPServer(("127.0.0.1", port), handler)
    except OSError:
        # Port still in use after kill attempt - find a free one
        server = HTTPServer(("127.0.0.1", 0), handler)
        port = server.server_address[1]

    url = f"http://localhost:{port}"
    print(f"\n Eval Viewer")
    print(f" _____")
    print(f" URL:          {url}")
    print(f" Workspace: {workspace}")
    print(f" Feedback:  {feedback_path}")
    if previous:
        print(f" Previous:  {args.previous_workspace} ({len(previous)} runs)")
    if benchmark_path:
        print(f" Benchmark: {benchmark_path}")
    print(f"\n Press Ctrl+C to stop.\n")

    webbrowser.open(url)

    try:
        server.serve_forever()
    except KeyboardInterrupt:
        print("\nStopped.")
        server.server_close()

if __name__ == "__main__":
    main()

```